Terraform for AWS

모듈 03 – Terraform 구성요소

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

MEGAZONE CLOUD

Topic

01 Resources

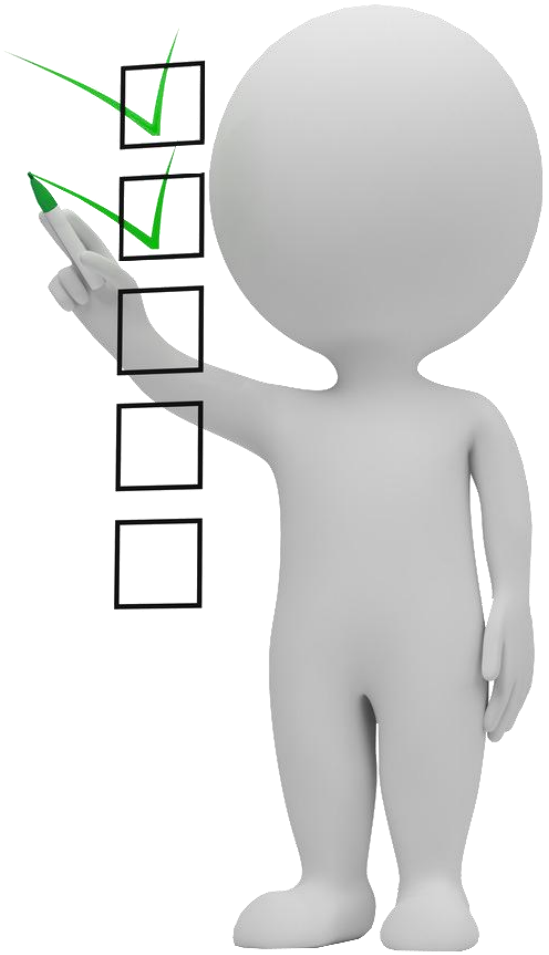
02 Data Source

03 Variables

04 Resources 에 대한 Meta-arguments

05 Resources 종속성

06 Output values



Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

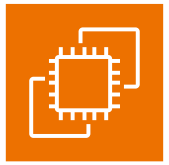
Resources 란 ?

- Resources는 Terraform을 사용하여 구성할 수 있는 인프라 요소 입니다.
  예) EC2, VPC, Amazon RDS, DynamoDB, ELB, Auto Scaling, EKS, S3 등

- Terraform은 각 AWS 서비스의 기본 API를 사용하여 리소스를 배포합니다.

```
resource “resource_type" “resource_name" {
  # Resource specific arguments
}
```

AWS Resources

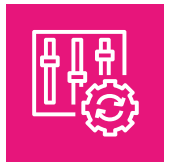


Amazon Elastic Compute Cloud (Amazon EC2)

Amazon Elastic Kubernetes Service (Amazon EKS)

Amazon Aurora

Amazon DynamoDB



AWS Config

Amazon Virtual Private Cloud (Amazon VPC)

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

리소스를 선언하는 구문

```
-- network /
     -- main.tf
     -- outputs.tf
     -- variables.tf
```

```
resource “resource_type" “resource_name" {
  # Resource specific arguments
}
```

- .tf file에 필요한 resource를 정의 합니다.
- resource block 은 인프라를 구성하는 단일 Object를 의미 합니다.
- resource type은 생성되는 리소스의 유형을 식별 합니다.
- resource type은 Terraform module 내에서 선언되는 provider에 따라 달라집니다.
- 모든 리소스 인수를 정의해야 하는 것은 아닙니다.

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

리소스 블록 예제

```
-- network /
      -- main.tf
      -- outputs.tf
      -- variables.tf
```

```
resource “aws_vpc" "vpc_example" {
    cidr_block                    = var.vpc_cidr  #Required argument
    instance_tanancy              = “default"
    enable_dns_hostname           = true
    tags = {
      Name  = “example-vpc”
    }
}
```

```
resource “aws_subnet" “public-subnet-1" {
    vpc_id                        = aws_vpc.vpc_example.id
    cidr_block                    = var.public-web-subnet-1-cidr
    avaliability_zone             = “ap-northeast-2a”
    map_public_ip_on_launch       = true
    tags = {
      Name = “Public_Subnet_1”
    }
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

리소스 속성을 참조하는 구문

-- main.tf

- resource 속성에 엑세스 하기 위해서 <resource_type>.<resource_name>.<attribute>를 사용함.
- VPC ID는 aws_vpc 블록의 자동 생성되는 개체 리소스 속성 입니다.

```
resource “aws_vpc" "vpc_example" {
    cidr_block                    = var.vpc_cidr  #Required argument
    instance_tanancy              = “default"
    enable_dns_hostname           = true
    tags = {
      Name  = “example-vpc”
    }
}

resource “aws_subnet" “public-subnet-1" {
    vpc_id                        = aws_vpc.vpc_example.id
    cidr_block                    = var.public-web-subnet-1-cidr
    avaliability_zone             = “ap-northeast-2a”
    map_public_ip_on_launch       = true
    tags = {
      Name = “Public_Subnet_1”
    }
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Resource block 정의시 주의사항

- 지정된 resource type 의 resource name은 모듈 내에서 고유해야 합니다.
- resource type은 사용자 정의가 아니라 공급자를 기반으로 합니다.

User-defined names are not allowed

```
resource “object_storage_bucket" "dev_bucket" {
  bucket          = "<unique_bucket_name>"
  tags = {
    Name                = “Mybucket”
    Environment         = “Dev”
  }
}
```

```
resource “aws_s3_bucket" "dev_bucket" {
  bucket          = "<unique_bucket_name>"
  tags = {
    Name                = “Mybucket”
    Environment         = “Dev”
  }
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Resource block 정의시 주의사항 (Cont)

- 지정된 resurce type 의 resource name은 모듈 내에서 고유해야 합니다.
- resource type은 사용자 정의가 아니라 공급자를 기반으로 합니다.
- **모든 구성 인수는 resource block 내에 묶어야 합니다.**

```
resource “aws_s3_bucket" "dev_bucket" {
  bucket          = "<unique_bucket_name>"
}

  object_lock_enabled  = true
```

```
Error: Unsupported argument

    On main.tf line 4:
    4: object_lock_enabled = true

An argument named “object_lock_enabled” is not expected here.
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Resource block 정의시 주의사항 (Cont)

- 지정된 resurce type 의 리소스 이름은 모듈 내에서 고유해야 합니다.
- resource type은 사용자 정의가 아니라 공급자를 기반으로 합니다.
- 모든 구성 인수는 resource block 내에 묶어야 합니다.
- **모든 필수 리소스 인수를 정의해야 합니다.**

```
resource “aws_ami" “example" {
  root_device_name     = “/dev/xvda“ #name 인수없음
}
```

```
resource " aws_ami" “example" {
  name                 = "<ami_name>"
  root_device_name     = “/dev/xvda”
}
```

```
Error: Missing required argument

   On main.tf line 9, in resource “aws_ami” “example”:
   9: resource “aws_ami” “example” {

The argument "name" is required, but no definition was found.
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Topic

01 Resources

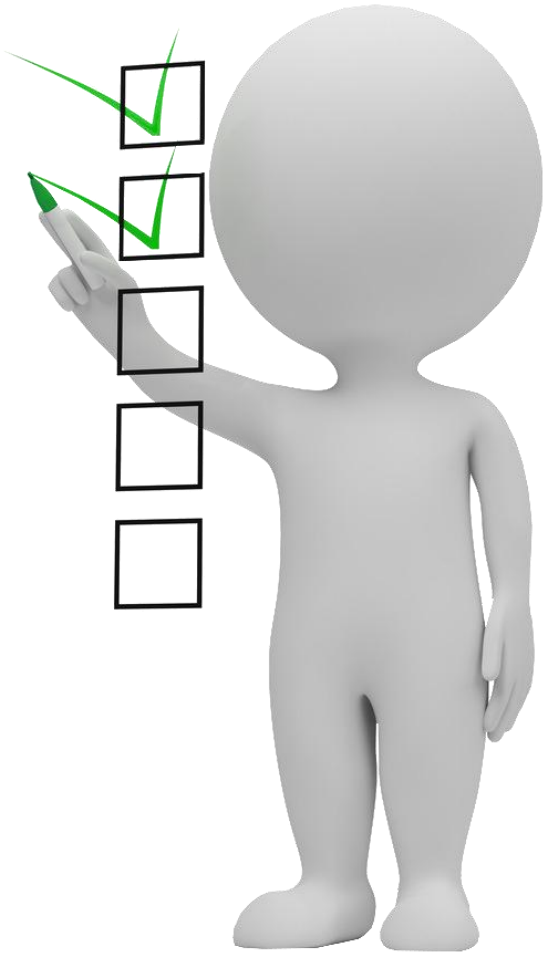
02 Data Source

03 Variables

04 Resources 에 대한 Meta-arguments

05 Resources 종속성

06 Output values



Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Data Source 란?

- Terraform 외부 리소스에 대한 정보를 가져오고 사용하여 Terraform 리소스를 설정하는데 사용
- 기존 클라우드 리소스의 다양한 속성을 검색하는데 사용
- 리소스 정의 내에서 사용할 수 있는 동적 및 컨텍스트 인식 속성을 제공
- 리소스를 생성하거나 관리하지 않고 리소스 구성을 알리는 정보를 검색

```
data "provider_type" "name" {
  # Configuration options for the data source
(filters)
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

사용사례 : 외부 리소스 속성에 액세스

- AWS 계정에서 특정 S3 버킷의 버킷 ID, ARN, 지역 및 도메인 이름과 같은 세부 정보를 가져와야 할 경우

```
# Data Source 정의
data "aws_s3_bucket" “mzc_bucket" {
  bucket = “mzc-log-bucket”
}
```

```
# Output 정의
output "bucket_id" {
  value = data.aws_s3_bucket.mzc_bucket.id
}
output "bucket_arn" {
  value = data.aws_s3_bucket.mzc_bucket.arn
}
output "bucket_region" {
  value = data.aws_s3_bucket.mzc_bucket.region
}
```

```
# 출력 결과
Outputs:
bucket_id = "mzc-log-bucket "
bucket_arn = "arn:aws:s3:::mzc-log-bucket"
bucket_region = “ap-northeast-2"
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

사용사례 : 원격 상태 파일의 데이터 사용

```
# Data Source 정의
data “terraform_remote_state" “network" {
  backend = “S3”
  config = {
    bucket = “tf-remotestate-vpc”
    key = “state”
    region = “ap-northeast-2”
  }
}
```

```
# 출력 결과
Apply complete! Resources: 0 added, 0 changed, 0
destroyed.
Outputs:
pri_subnet_id = "subnet-03a33174741fd9cb4"
pub_subnet_id = "subnet-0fad314b1f1e652bc"
```

- 원격에 저장된 Terraform State 파일에서 정보를 쿼리하기 위해 사용

```
# Output 정의
output “pub_sunet_id" {
  value = data.terraform_remote_state.network.outputs.subnet_pub_id
}
output “pri_sunet_id" {
  value = data.terraform_remote_state.network.outputs.subnet_pri_id
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

사용사례 : 데이터 소스를 사용하여 AWS 암호에 액세스

- Secrets Manager에 민감한 인증 정보를 저장하고 Amazon RDS에서 사용하기

```
# Data Source 정의
data "aws_secretsmanager_secret" "mydb_secret" {
  arn = "arn:aws:secretsmanager:eu-central-1:532199187081:secret:pg_db-dKuRrd"
}

data "aws_secretsmanager_secret_version" "mydb_secret_version" {
  secret_id = data.aws_secretsmanager_secret.mydb_secret.id
}
```

```
# Resource에 해당 값에 액세스 설정
resource "aws_db_instance" "my_db" {
  . . .
  instance_class = "db.t2.micro"
  username = jsondecode(data.aws_secretsmanager_secret_version.mydb_secret_version.secret_string).username
  password = jsondecode(data.aws_secretsmanager_secret_version.mydb_secret_version.secret_string).password
  tags = {
    Name = "ExampleDB"
  }
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

사용사례 : 데이터 소스를 사용하여 동적 구성 만들기

- 클라우드 공급자로 부터 중요한 세부정보를 검색하여 현재 Terraform 프로젝트의 실제 변경 사항에 맞게 구성을 조정

```
# Data Source 정의
data "aws_subnets" "my_subnets" {
  filter {
    name = "vpc-id"
    values = ["vpc-0cb1aa34b173b1bb6"]
  }
}
```

```
# Resource에 해당 값에 액세스 설정
resource "aws_instance" "my_vm_2" {
  for_each = toset(data.aws_subnets.my_subnets.ids)
  ami = var.ami //Ubuntu AMI
  instance_type = var.instance_type

  subnet_id = each.key

  tags = {
    Name = var.name_tag,
  }
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Topic

01 Resources

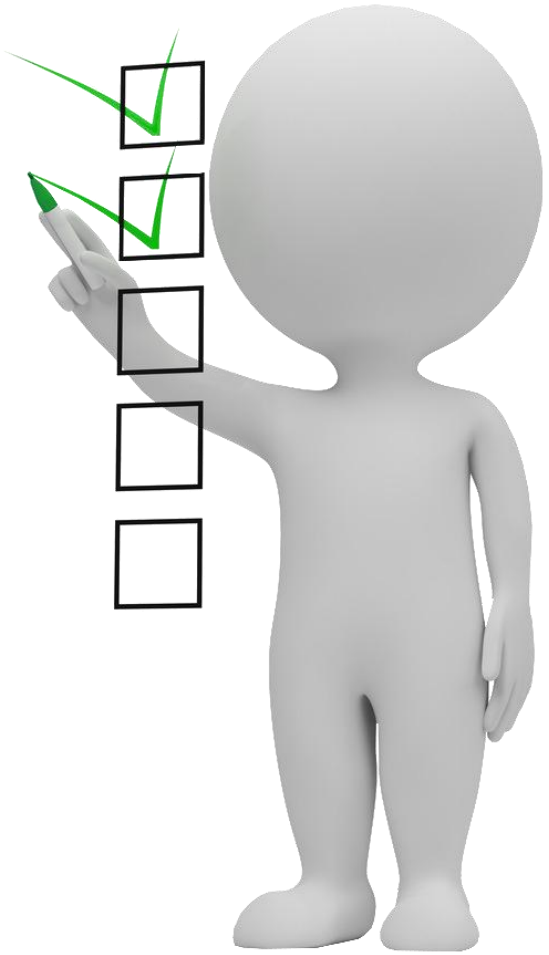
02 Data Source

03 Variables

04 Resources 에 대한 Meta-arguments

05 Resources 종속성

06 Output values



Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Variable 이란?

- 변수를 소스 코드를 변경하지 않고 구성을 매개변수화 합니다.
- 변수를 사용하면 run time에 리소스 속성에 값을 할당 할 수 있습니다.
- 변수는 소스 코드와 값 할당을 분리 합니다.

```
resource “aws_instance" “example-vm-1" {
  ami                   = “ami-005e54dee72cc1d00“
  instance_type         = "t3.micro"
  availability_zone     = “us-west-2a”
}
```

변수가 없으면 resource 인수는 config 에 하드코딩 됩니다.

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

입력 변수(variable)를 선언하는 구문

```
-- server /
     -- main.tf
     -- outputs.tf
     -- variables.tf
```

Syntax

```
variable "variable_name" {
    description          = "<variable description>"
    type                 = <variable_type>
    default              = "<default value for variable>"
    sensitive            = true
}
```

Example: Variable for a EC2 instance type

```
variable “instance_type" {
    description = “web server instance type"
    type        = string
    default     = “t2.micro"
    sensitive     = true
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Variabel arguments: type

```
-- server /
     -- main.tf
     -- outputs.tf
     -- variables.tf
```

```
variable "variable_name" {
    #Arguments
    type                    = <variable_type>
    default                 = "<default value for variable>"
    description             = "<variable description>"
    sensitive               = true
}
```

```
variable “instance_type” {
  type            = string
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Variabel arguments: default

```
-- server /
    -- main.tf
    -- outputs.tf
    -- variables.tf
```

m

v

변수에 대해 설정된 값이 없으면 기본 인수에 지정된 값이 사용됩니다.

```
$terraform plan
Terraform will perform the following actions:

 # aws_instance.mywebserver will be created
  + resource “aws_instance" “mywebserver" {
      + ami                          = “ami-0dcc1e21636832c5d"
      + tenancy                      = “host"
      + instance_type                = “t3.micro”
..

Plan: 1 to add, 0 to change, 0 to destroy.
```

m

```
resource “aws_instance" "mywebserver" {
  ami                    = “ami-0dcc1e21636832c5d"
  tenancy                = “host”
  instance_type          =  var.instance_type
  //No value assigned for the ec2 instance
}
```

v

```
variable “instance_type" {
type      = string
default   = “t3.micro"
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Variabel arguments: description

```
-- server /
     -- main.tf
     -- outputs.tf
     -- variables.tf
```

```
variable "variable_name" {
     type          = <variable_type>
     default       = “<default value>"
     description   = “<variable description>”
     sensitive     = true
}
```

```
variable "bucket_region" {
     type          = string
     description = “Specify the instanc type.”
}
```

```
$terraform plan

var.instance_type
  Specify the instance type.

  Enter a value:
```

변수에 값이 할당되지 않은 경우 런타임에 설명이 표시됩니다.

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Variabel arguments: sensitive

```
-- server /
     -- main.tf
     -- outputs.tf
     -- variables.tf
```

- sensitive에 허용되는 값은 true 입니다.
- true로 설정하면 run time 중 값을 (sensitive) 로 표시 합니다.

```
$terraform plan
Terraform will perform the following actions:

  # some_resource.a will be created
  + resource "some_resource" “foo" {
      + username    = (sensitive)
      + password    = (sensitive)
    }

Plan: 1 to add, 0 to change, 0 to destroy.
```

```
variable "variable_name" {
     type            = <variable_type>
     default         = “<default value>”
     description     = “<variable description>”
     sensitive       = true
}
```

```
variable "user_information" {
     type = object({
         username    = string
         password = string
       })
  sensitive = true
}

resource "some_resource" "foo" {
     username    = var.user_information.username
     password    = var.user_information.password
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

입력 변수(input variable)를 참조하는 구문

```
-- server /
    -- main.tf
    -- outputs.tf
    -- variables.tf
```

m

v

m

```
resource “aws_instance" “myinstance" {
  ami                 = "ami-0dcc1e21636832c5d"
  instance_type       = var.instance_type
  tenancy             = “host"
}
```

v

```
variable “instance_type” {
     type            = string
     description     = “ec2 instance type”
     default         = “t3.micro”
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

variables 값을 할당하는 다양한 방법

- **.tfvars 파일**: 변수 집합을 빠르게 전환하고 버전을 관리하는 데 유용합니다.
- **CLI –var 옵션** : 간단한 파일에서 빠른 매체를 실행할 때 유용합니다.
- **환경변수**: 스크립트 및 파이프라인에서 유용합니다.
- **CLI 프롬프트**: 위 중 하나를 통해 필수 변수를 설정하지 않은 경우.

```
# .tfvars file (Recommended method)
$ terraform apply -var-file my-vars.tfvars

# CLI options
$ terraform apply -var project_id="my-project"

# enviroment variables
$ export TF_VAR_instane_type="t3.small"
$ terraform apply

# If using terraform.tfvars
$ terraform apply
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

변수(variable)에 값을 할당하는 방법(권장)

terraform.tfvars file을 사용

```
-- server /
     -- main.tf
     -- outputs.tf
     -- terraform.tfvars
     -- variables.tf
```

t

v

v

```
variable “instance_type" {
    type  = string
}

variable “ami_id” {
     type = string
}
```

t

```
instance_type       = “t3.small"
ami_id              = "ami-0dcc1e21636832c5d"
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

변수(variable)에 값을 할당하는 방법

tf apply 실행시 -var 옵션 사용

```
-- server /
     -- main.tf
     -- outputs.tf
     -- my-vars.tf
     -- terraform.tfvars
     -- variables.tf
```

M
T
v

T

```
instance_type        = “t3.small"
ami_id               = "ami-0dcc1e21636832c5d"
```

M

```
instance_type        = “t3.small"
ami_id               = "ami-0dcc1e21636832c5d"
```

```
$cd /server
$terraform apply –var instance_type=“t3.large”

$tf apply -var-file my-vars.tf
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

필수 변수(variable)가 없을 경우 값을 할당하는 방법

run time 시 변수 값 입력

```
-- server /
    -- main.tf
    -- outputs.tf
    -- variables.tf
```

M

V

```
$terraform plan

var.instnace_type
  Set the instance tpye to the ec2.

  Enter a value:
```

M

```
resource “aws_instance" “myinstance" {
  ami                 = "ami-0dcc1e21636832c5d"
  instance_type       = var.instance_type
  tenancy             = “host"
}
```

V

```
variable “instance_type” {
     type            = string
     description     = “ec2 instance type”
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

variable 의 값에 대해 유효성을 검사

```
variable “iam_instnace_profile" {
  type        = string
  description = "Set the instance profile to the ec2."
  validation {
    condition = contains([“~~~~"], var.iam_instance_profile)
    error_message = “Invalid value for variable."
  }
}
```

```
var.ec2_iam_role_name
  Set the iam role to ec2.
  Enter a value: cloud-manager
aws_instance.ec2_iam_role_name Refreshing state...

╵
│ Error: Invalid value for variable
│
│   on main.tf line 1:
│    1: variable “iam_instance_profile" {
│
│ The ec2_iam_role_name value must be a valid IAM role name, starting with “svc-”.
│
│ This was checked by the validation rule at main.tf:4,3-13.
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

필요한 경우에만 매개변수화 합니다.

```
-- server /
     -- main.tf
     -- outputs.tf
     -- terraform.tfvars
     -- variables.tf
```

- 각 instance 또는 environment 마다 변경해야 하는 값만 변수화 하세요.
- 기본값이 있는 변수를 변경하면 이전 버전과 호환됩니다.
- 변수를 제거하면 이전 버전과 호환되지 않습니다.

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

.tfvars file

```
-- server /
     -- main.tf
     -- outputs.tf
     -- my-vars.tf
     -- terraform.tfvars
     -- variables.tf
```

```
instance_type     = “t3.small"
ami_id            = "ami-0dcc1e21636832c5d"
```

Command-line options 은 임시적이며 source control 에서 확인 할 수 업습니다.

```
$cd /server
$terraform apply –var instance_type=“t3.large”

$tf apply -var-file my-vars.tf
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

변수는 설명이 포함된 이름 지정

```
-- server /
     -- main.tf
     -- outputs.tf
     -- my-vars.tf
     -- terraform.tfvars
     -- variables.tf
```

```
variable "ram_size_gb" {
    type                = number
    description         = "RAM size in GB"
}
```

```
variable "ram_size" {
    type                = string
    description         = "RAM size in GB."
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

의미 있는 descriptions 제공

```
-- server /
     -- main.tf
     -- outputs.tf
     -- my-vars.tf
     -- terraform.tfvars
     -- variables.tf
```

```
variable "instance_type" {
    type                    = string
    default                 = "t3.small"
    description             = "Specify the instance type."
}


variable "mytype" {
    type                    = string
    default                 = "t3.small"
    description             = "Specify the type."
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Topic

01 Resources

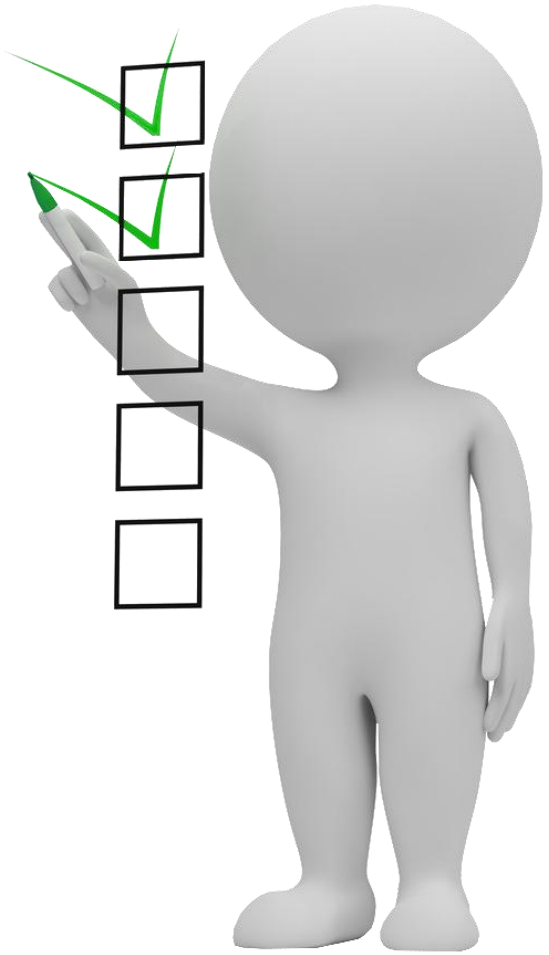
02 Data Source

03 Variables

04 Resources 에 대한 Meta-arguments

05 Resources 종속성

06 Output values



Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Terraform Meta-Argument

count	count 에 할당된 값에 따라 여러 인스턴스를 생성.
for_each	map 또는 string 집합에 따라 여러 리소스 인스턴스를 생성.
depends_on	명시적으로 종속성을 지정.
lifecycle	resource 수명 주기 정의.
provider	기본값이 아닌 공급자 구성 선택.

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

count: 같은 종류의 여러 리소스 생성

Redundant code

```
resource “aws_instance" "dev_VM1" {
 …
  tags = {
    Name = "dev_VM1”
}

resource “aws_instance" "dev_VM2" {
 …
  tags = {
    Name = "dev_VM2”
}

resource “aws_instance" "dev_VM3" {
 …
  tags = {
    Name = "dev_VM3”
}
```

동일한 타입의 instance 3개 생성

```
resource “aws_instance" "Dev_VM" {
  count                          = 3
  tags = {
    Name  = "dev_VM${count.index + 1}"
    #other required arguments
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

for_each: 고유한 값을 가진 동일한 유형의 여러 리소스

Redundant code

```
resource “aws_subnet" “public-subnet-1" {
 vpc_id              = var.vpc_id
 availability_zone = "us-west-2a“
 ..
resource “aws_subnet" “public-subnet-2" {
 vpc_id              = var.vpc_id
 availability_zone = "us-west-2b“
..
resource “aws_subnet" “public-subnet-3" {
 vpc_id              = var.vpc_id
 availability_zone = "us-west-2c“
..
```

pubsub-us-west-2a,
pubsub-us-west-2b,
pubsub-us-west-2c,
이름을 가지고 있는 3개의 서브넷 생성

```
resource “aws_subnet" “public-subnet" {
  for_each          = toset( ["us-west-2a", " us-west-2b", " us-west-2c"] )
  availability_zone = each.value

  tags = {
    Name            = “pubsub-${each.value}"
    #other required arguments
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Topic

01 Resources

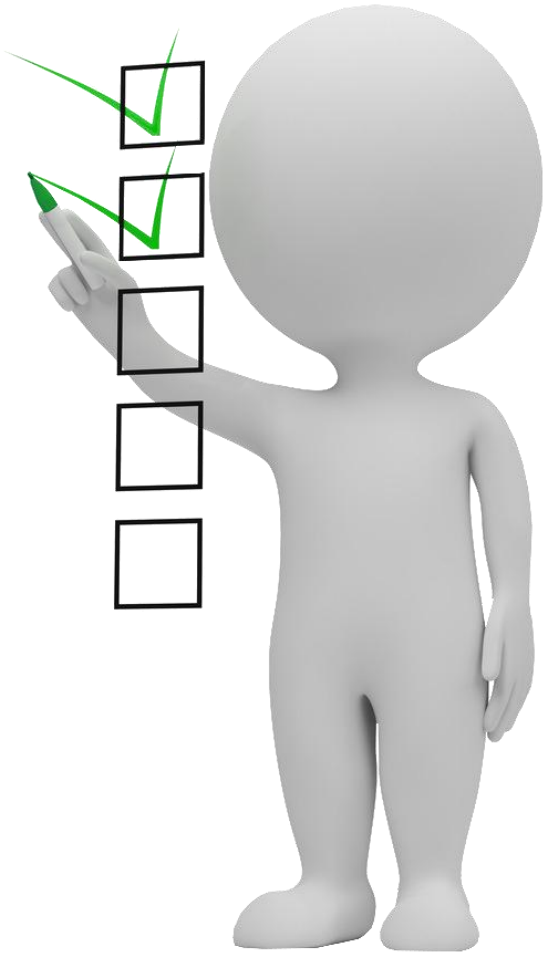
02 Data Source

03 Variables

04 Resources 에 대한 Meta-arguments

05 Resources 종속성

06 Output values

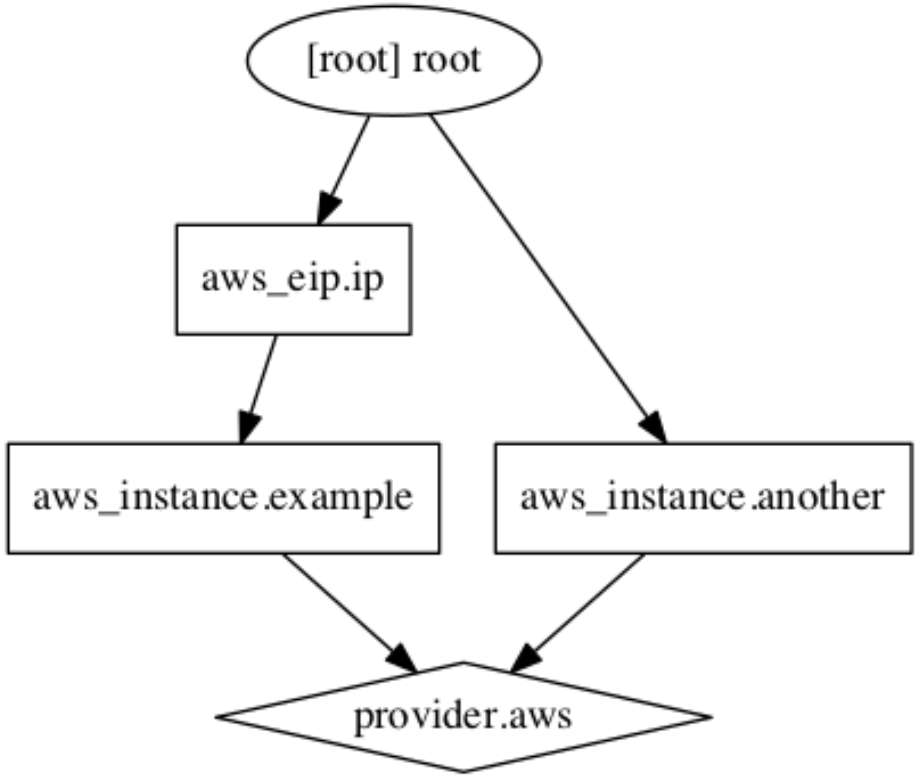


Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Dependency graph

- Terraform 구성에서 빌드됩니다.
- Denpendency를 사용할 때 Terraform은 run time 중에 리소스의 속성에 변수의 값을 바인딩 합니다.
- 올바른 작업 순서를 결정합니다.

```
Prescott@Prescottui-Macbook-Pro /p/t/terraform-101> terraform graph
digraph {
        compound = "true"
        newrank = "true"
        subgraph "root" {
                "[root] aws_eip.ip" [label = "aws_eip.ip", shape = "box"]
                "[root] aws_instance.another" [label = "aws_instance.another", shape = "box"]
                "[root] aws_instance.example" [label = "aws_instance.example", shape = "box"]
                "[root] provider.aws" [label = "provider.aws", shape = "diamond"]
                "[root] aws_eip.ip" -> "[root] aws_instance.example"
                "[root] aws_instance.another" -> "[root] provider.aws"
                "[root] aws_instance.example" -> "[root] provider.aws"
                "[root] root" -> "[root] aws_eip.ip"
                "[root] root" -> "[root] aws_instance.another"
        }
}

Prescott@Prescottui-Macbook-Pro /p/t/terraform-101>
```



Graphviz로 시각화 한 terraform graph

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Resource dependencies

Terraform 은 두 가지 종류의 종속성을 처리할 수 있습니다.

Implicit denpendency	Explicit depndency
Terraform은 알려진(known) 종속성은 자동으로 감지 됩니다.	Terraform은 알려지지 않은 (unknown) 종속성은 명시적으로 구성해야 합니다.

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Implicit(암시적) 리소스 종속성은 자동으로 처리

```
resource "aws_vpc" "my_vpc" {
  cidr_block = "172.16.0.0/16"

  tags = {
    Name = "tf-example"
  }
}

resource "aws_subnet" "my_subnet" {
  vpc_id            = aws_vpc.my_vpc.id
  cidr_block        = "172.16.10.0/24"
  availability_zone = "us-west-2a"

  tags = {
    Name = "tf-example"
  }
}
```

aws_vpc 인수의 my_vpc에 대한 참조는 암시적(알려진) 종속성을 생성합니다.

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

terraform apply: 암시적(Implicit) 종속성

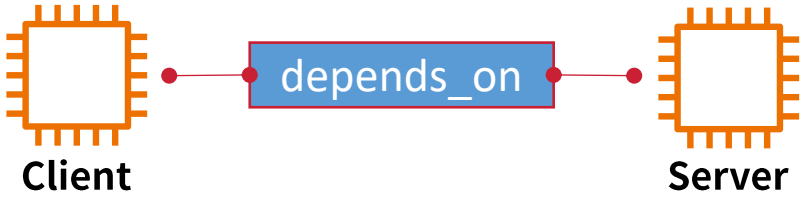
Terraform은 VPC를 먼저 생성 합니다.

implicit(암시적) 종속성이 충족되면(vpc is create) subnet을 생성.

```
$terraform apply

aws_vpc.my_vpc: Creating...
aws_vpc.my_vpc: Still creating... [10s elapsed]
aws_vpc.my_vpc : Still creating... [20s elapsed]
aws_vpc.my_vpc : Still creating... [30s elapsed]
aws_vpc.my_vpc : Creation complete after 32s [id=e973d950dd4a/aws_vpc/my_vpc]


aws_subnet.my_subnet: Creating...
aws_subnet.my_subnet: Still creating... [10s elapsed]
aws_subnet.my_subnet : Creation complete after 13s [id=e973d950dd4a/my_vpc/my_subnet]
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Explicit(알수없는) 종속성은 depends_on 인수 사용




```
resource "resource_type" "resource_name" {

..
depends_on = [<resource_type>.<resource_name>]
}
```

클라이언트 인스턴스는 서버 인스턴스가 생성된 경우에만 생성할 수 있습니다.

```
resource “aws_instance" "client" {
 ...
 depends_on = [aws_instance.server]
}

resource “aws_instance" "server" {

 #All required configuration options
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

terraform apply : 명시적(Explicit) 종속성

client 생성 전 server 생성

explicit(명시적) 종속성으로 client은 server가 생성된 후 생성

```
$terraform apply

aws_instance.server: Creating...
aws_instance.server: Still creating... [10s elapsed]
aws_instance.server: Creation complete after 12s [id=projects/qwiklabs-gcp-01-
e973d950dd4a/zones/us-central1-a/instances/server]

aws_instance.client: Creating...
aws_instance.client: Still creating... [10s elapsed]
aws_instance.client: Creation complete after 13s [id=projects/qwiklabs-gcp-01-
e973d950dd4a/zones/us-central1-a/instances/client]
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Topic

01 Resources

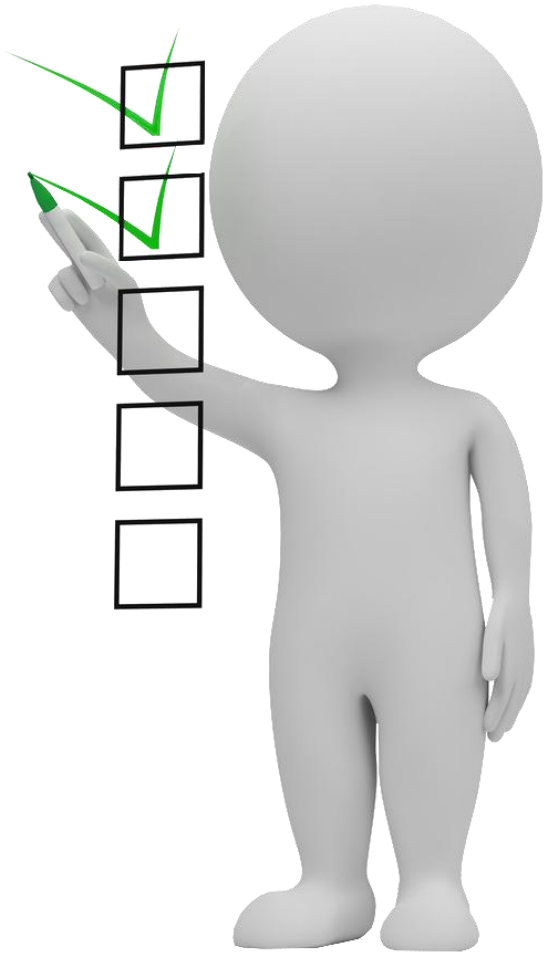
02 Data Source

03 Variables

04 Resources 에 대한 Meta-arguments

05 Resources 종속성

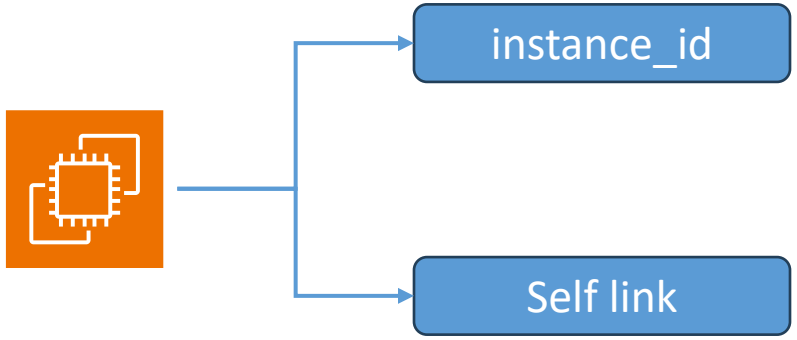
06 Output values



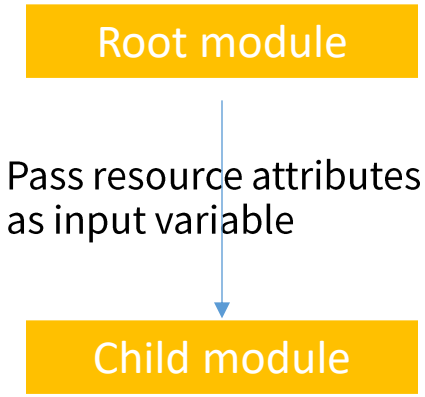
Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Output values 란?

Output values 은 Terraform 사용자에게 리소스에 대한 정보를 노출합니다.



Print resource attributes



Pass resource attributes

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

output을 사용하여 리소스 속성 출력

```
-- server /
    -- main.tf
    -- outputs.tf
    -- variables.tf
```

M

O

M

```
resource "aws_instance" "web-server" {
…
}
```

O

```
output "instance_public_ip" {
  description = "Public IP address of the EC2 instance"
//value   = <resource_type>.<resource_name>.<attribute>
  value   = aws_instance.web-server.public_ip
```

사용자가 제공한 입력 대신 출력 값을 사용하는 것이 좋습니다.

```
aws_instance.web-server: Creating...
aws_instance.web-server : Creating complete after 1s []

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

instance_public_ip = "35.165.236.5"
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

terraform output

```
-- server /
     -- main.tf
     -- outputs.tf
     -- variables.tf
```

```
resource “aws_instance" “this" {
  …
}

output instance_id {
     value = aws_instance.this.id
}
output public_dns {
     value = aws_instance.this.public_dns
}
```

Query all output values.

```
$terraform output
instance_id = "i-1234567890abcdef0"
public_dns = “ec2-52.27.125.25.us-west-2.compute.amazonaws.com"
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Output useful information

```
-- server /
     -- main.tf
     -- outputs.tf
     -- variables.tf
```

```
output "lb_url" {
  description = "URL of load balancer"
  value       = "http://${module.elb_http.elb_dns_name}/"
}

output "web_server_count" {
  description = "Number of web servers provisioned"
  value       = length(module.ec2_instances.instance_ids)
}
```

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
Outputs:
lb_url = "http://lb-5YI-project-alpha-dev-2144336064.us-east-1.elb.amazonaws.com/"
web_server_count = 4
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

의미 있는 descriptions 제공

```
-- server /
     -- main.tf
     -- outputs.tf
     -- variables.tf
```

```
output "lb_url" {
  description = "URL of Application load balancer"
  value       = "http://${module.elb_http.elb_dns_name}/"
}
```

```
output "lb_url" {
  description =  " My-URL"
  value       = "http://${module.elb_http.elb_dns_name}/"
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

모든 output은 outputs.tf 파일로

```
-- server /
    -- main.tf
    -- outputs.tf
    -- variables.tf
```

```
output "lb_url" {
  description = "URL of load balancer"
  value       = "http://${module.elb_http.elb_dns_name}/"
}

output "web_server_count" {
  description = "Number of web servers provisioned"
  value       = length(module.ec2_instances.instance_ids)
}
```

```
resource "aws_instance" "web" {
  ami                     = "ami-a0cfeed8"
  instance_type           = "t2.micro"
  user_data               = file("init-script.sh")
  ...
  }
}

output "web_server_count" {
  description = "Number of web servers provisioned"
  value       = length(module.ec2_instances.instance_ids)
}
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

민감한(Sensitive) 출력 표시

```
-- server /
     -- main.tf
     -- outputs.tf
     -- variables.tf
```

```
output "sql_user_password" {
 value       = aws_sql_user.users.password
 description = "The password for sql user."
 sensitive   = true
}
```

민감한 값은 terraform plan 또는 terraform apply의 출력에서 “sensitive” 키워드로 마스킹 됩니다.

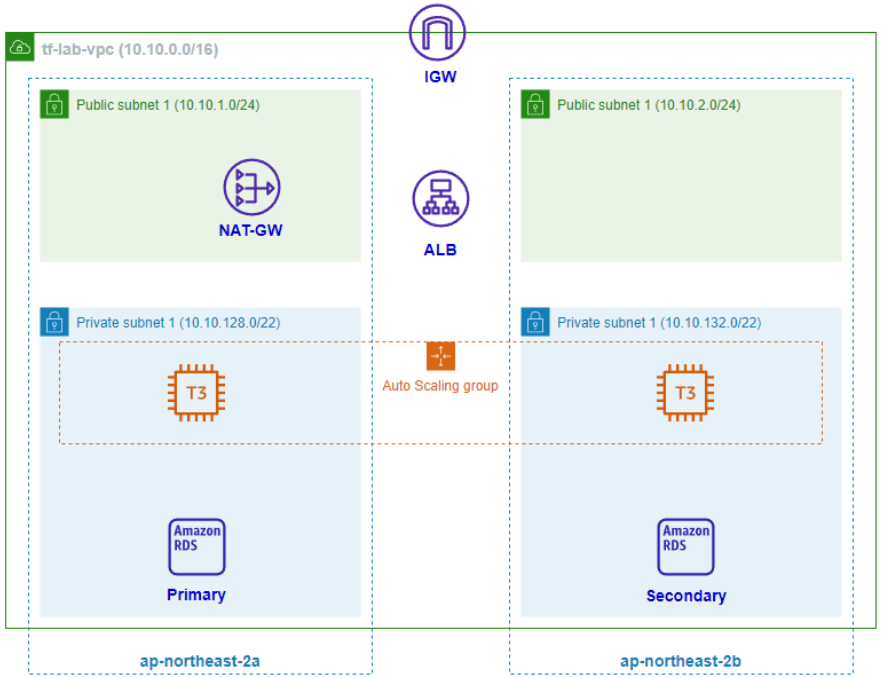
```
$terraform plan
Terraform will perform the following actions:

  # some_resource.a will be created
  + resource “aws_sql_user" "users" {
      + password    = (sensitive)
    }

Plan: 1 to add, 0 to change, 0 to destroy.
```

Copyright 2024 © MEGAZONE CLOUD CORP. ALL RIGHT RESERVED.

Lab : Input 변수를 이용한 리소스 배포하기

- **input valiable 을 사용한 Resource 배포**
  - VPC Resource 작성
  - EC2 Resource(ALB, ASG 포함) 작성
  - Amazon RDS Resource 작성

```
-- Multi-Tier /
   -- main.tf
   -- outputs.tf
   -- variables.tf
   -- terraform.tfvars
```



Public Route Table

Destination	Target
10.10.0.0/16	Local
0.0.0.0/0	IGW

Private Route Table 1

Destination	Target
10.10.0.0/16	Local
0.0.0.0/0	NAT-GW

Private Route Table 2

Destination	Target
10.10.0.0/16	Local
0.0.0.0/0	NAT-GW

소요시간 : **240분 ~**

Thank you